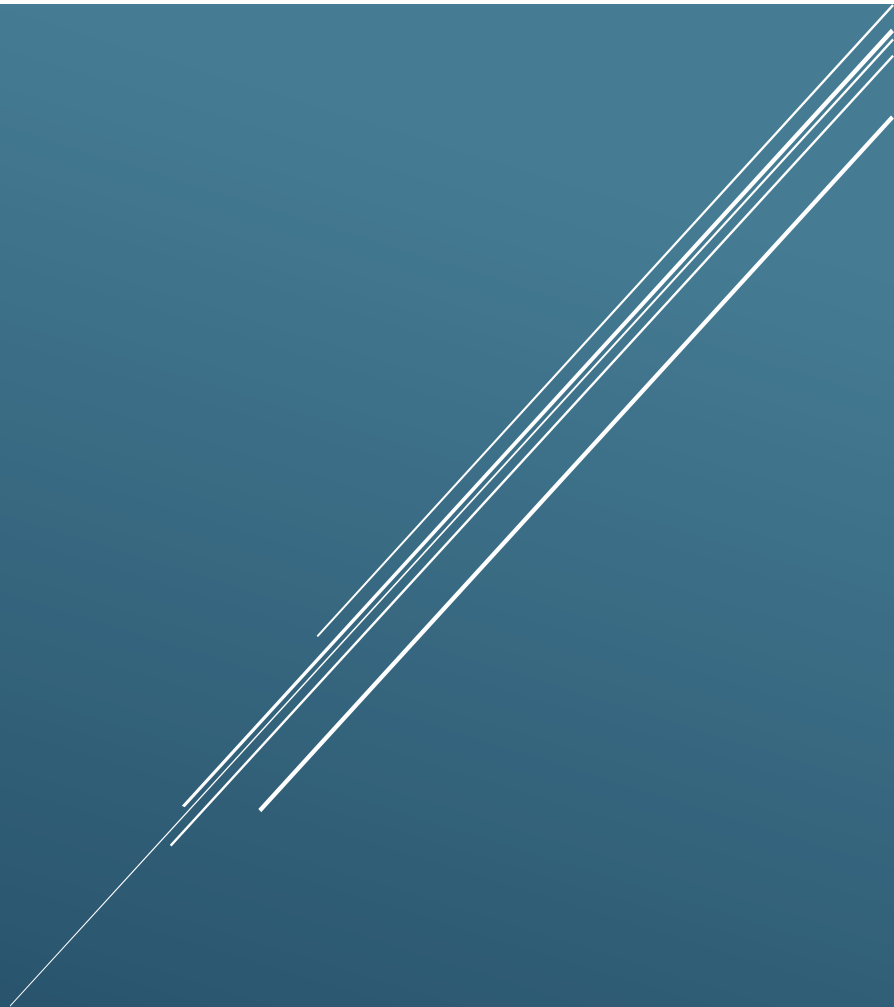




MEMORIA PROYECTO INTERMODULAR

Creado por: Yon, Ian y Carlos



Contenido

Memoria Proyecto Intermodular	2
Introducción / Introduction	2
Programación	2
UML Diagrama de Clases	2
Interfaz Gráfica y Lógica de la Aplicación	2
Mapa de Navegación	3
Pruebas de Software (Testing)	12
Base de Datos	12
Diseño ER	12
Más Información	13
Lenguaje de Marcas	14
Botón inglés	16
Botón Dark Mode (Modo oscuro)	18
Prototipos	19
Mapa Navegación	19
Entornos de desarrollo	19
Implementación y Estructura del Proyecto	22
Dificultades y Soluciones Técnicas	23
Selección de Colores y Temas	24
Reparto de Tareas y Trabajo en Equipo	24
Despliegue y Sistemas Informáticos	25
Itinerario Personal para la Empleabilidad	26
Conclusiones y Trabajo Futuro	27
Enlace al proyecto	27



Memoria Proyecto Intermodular

Introducción / Introduction

ES

En el presente documento se detalla el análisis, diseño e implementación de GYMSTATS, una aplicación de escritorio orientada a la gestión integral de centros deportivos. Su objetivo principal es digitalizar y centralizar los procesos operativos del gimnasio mediante una solución tecnológica robusta y escalable. El sistema abarca tres áreas clave: la administración de clientes y sus membresías, la organización del personal estructurada por perfiles (recepcionistas, limpiadores, entrenadores y administradores), y la coordinación de la oferta deportiva, gestionando las clases, las zonas de entreno y un buzón de contacto web. A continuación, se desglosa la arquitectura técnica del proyecto, partiendo del diseño conceptual con el Modelo Entidad-Relación hasta llegar a su implementación final en PostgreSQL, incluyendo la programación de reglas de negocio mediante vistas, funciones y triggers.

EN

This document details the analysis, design, and implementation of GYMSTATS, a desktop application aimed at the comprehensive management of sports centers. Its main objective is to digitize and centralize the operational processes of the gym through a robust and scalable technological solution. The system covers three key areas: the administration of clients and their memberships, the organization of staff structured by profiles (receptionists, cleaners, trainers, and administrators), and the coordination of the sports offering, managing classes, training zones, and a web contact inbox. Below, the technical architecture of the project is broken down, starting from the conceptual design with the Entity-Relationship Model up to its final implementation in PostgreSQL, including the programming of business rules through views, functions, and triggers.

Programación

UML Diagrama de Clases

[Click aquí](#) para ver el UML mejor.

Interfaz Gráfica y Lógica de la Aplicación

La interfaz de nuestra aplicación ha sido desarrollada en Java utilizando la biblioteca Swing para la creación de ventanas y paneles. Para modernizar el aspecto visual y adaptarlo a la paleta de colores de GymStats, hemos implementado el tema FlatDarkLaf. La estructura del código se divide lógicamente en clases de modelo, ventanas de navegación principal y formularios emergentes para la manipulación directa de los datos.

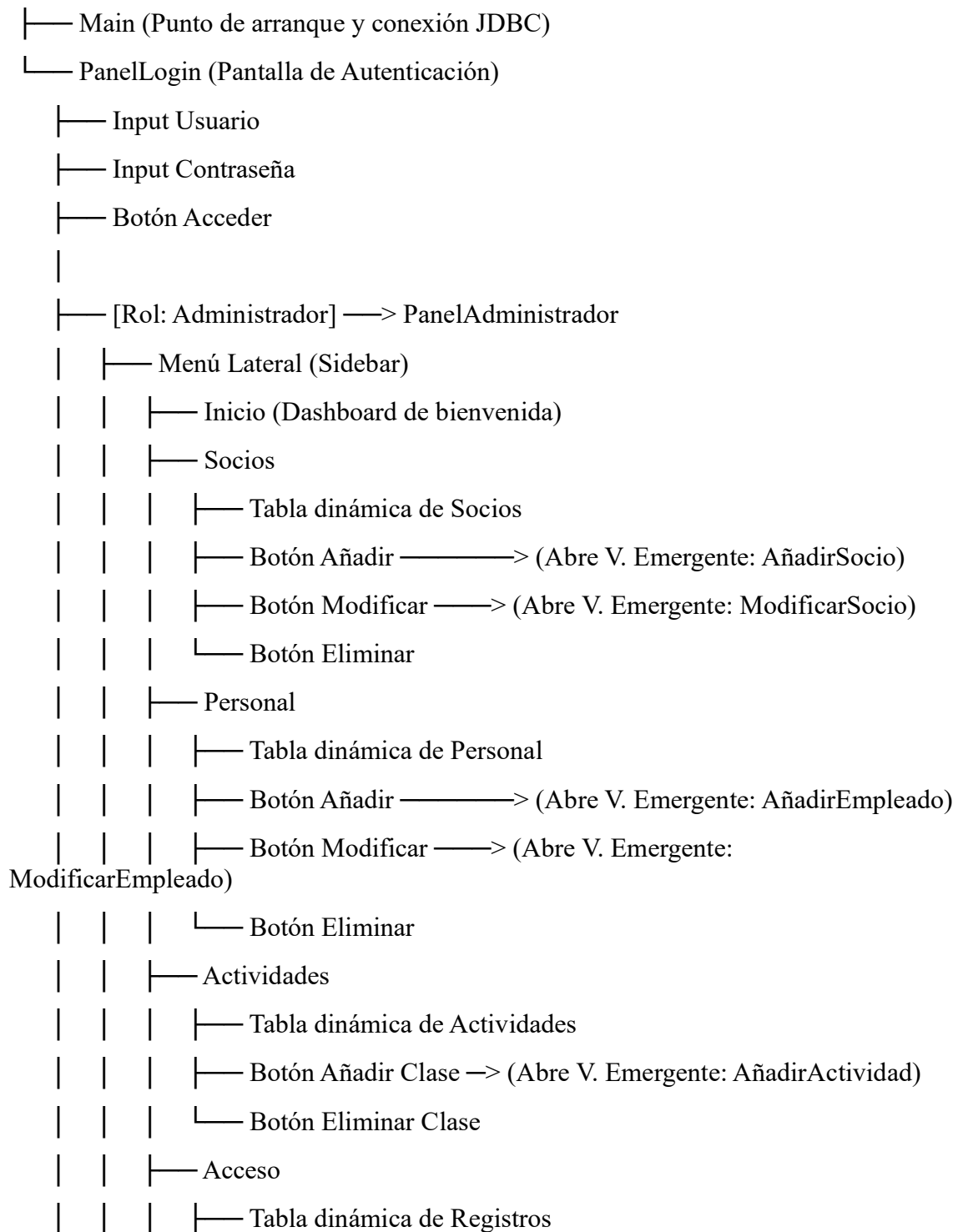


Existen 3 usuarios en la aplicación los administradores que pueden ver todo, los empleados que no pueden ver especificaciones de los otros empleados y por último los usuarios normales que podrán ver cuantos días de subscripción les quedan y el IMC.

Se seguirá un modelo vista controlador para el diseño de la aplicación.

Mapa de Navegación

GymStats Desktop App (Java Swing)





- | | | └─ Botón Modificar ──> (Abre V. Emergente: ModificarAcceso)
- | | └─ Contacto
- | | └─ Tabla dinámica de Mensajes Web
- | | └─ Doble clic en fila ─> (Abre V. Emergente: Leer Mensaje Completo)
- |
- | └─ [Rol: Empleado] ──> PanelEmpleado
- | | └─ Menú Lateral (Sidebar)
- | | | └─ Inicio
- | | | └─ Socios (Opciones CRUD limitadas)
- | | | | └─ Botón Añadir ──────────> (Abre V. Emergente: AñadirSocio)
- | | | | └─ Botón Modificar ──> (Abre V. Emergente: ModificarSocio)
- | | | └─ Actividades
- | | | | └─ Botón Añadir Clase ─> (Abre V. Emergente: AñadirActividad)
- | | | └─ Acceso
- | | | | └─ Botón Modificar ──> (Abre V. Emergente: ModificarAcceso)
- |
- | └─ [Rol: Cliente] ──> PanelUsuario (Perfil Personal)
- | | └─ Tarjeta Estado Membresía
- | | | └─ Días de acceso restantes
- | | | └─ Barra de progreso visual (JProgressBar)
- | | └─ Tarjeta IMC Actual
- | | | └─ Valor IMC y Estado (Bajo peso / Normal / Sobrepeso)
- | | └─ Tarjeta Calculadora
- | | | └─ Input Peso (kg)
- | | | └─ Input Altura (cm)
- | | | └─ Botón Actualizar ──────────> (Actualiza perfil e IMC en BD)

Configuración Base y Modelos de Datos

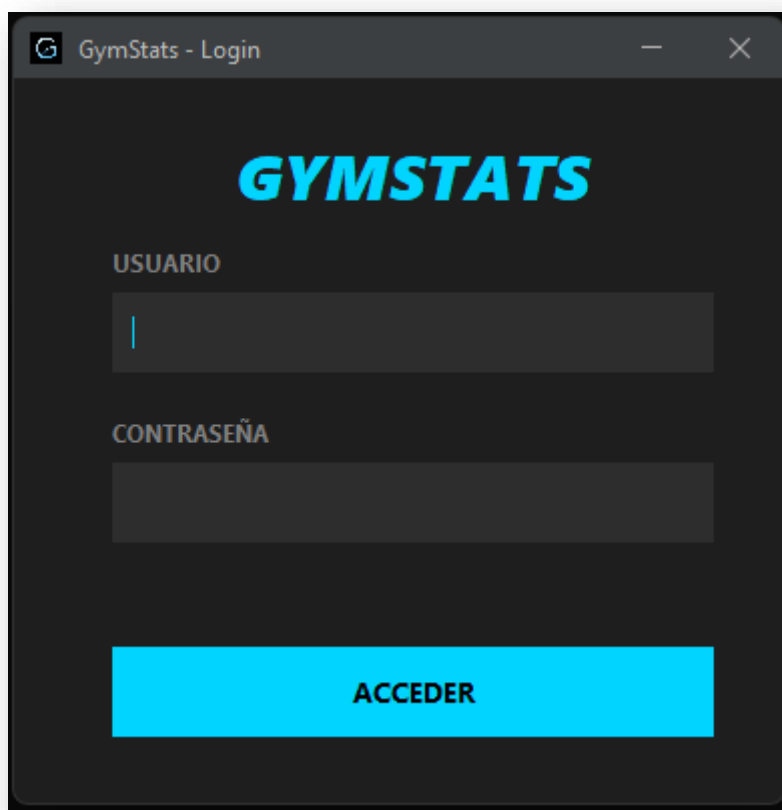
El archivo Main actúa como el núcleo de arranque. Su función es establecer el estilo visual, configurar la conexión a la base de datos PostgreSQL mediante el driver JDBC y lanzar la pantalla inicial. Por otro lado, contamos con clases de modelo como Usuario, Cliente y Membresia. Estas clases funcionan como representación orientada a objetos de



las tablas de nuestra base de datos, almacenando los atributos en memoria para facilitar su manejo durante la ejecución del programa.

Autenticación y Control de Accesos

El PanelLogin es la primera barrera de seguridad de la aplicación. Su lógica consiste en recoger las credenciales del usuario y ejecutar consultas secuenciales en la base de datos para verificar a qué grupo pertenece la persona que intenta acceder. Dependiendo de si los datos coinciden con un Cliente, un Administrador o un Empleado, el sistema instanciará y abrirá un panel de control distinto. Además, si el usuario es un cliente, este panel automatiza el registro de su hora de entrada en la tabla de accesos de forma completamente transparente.

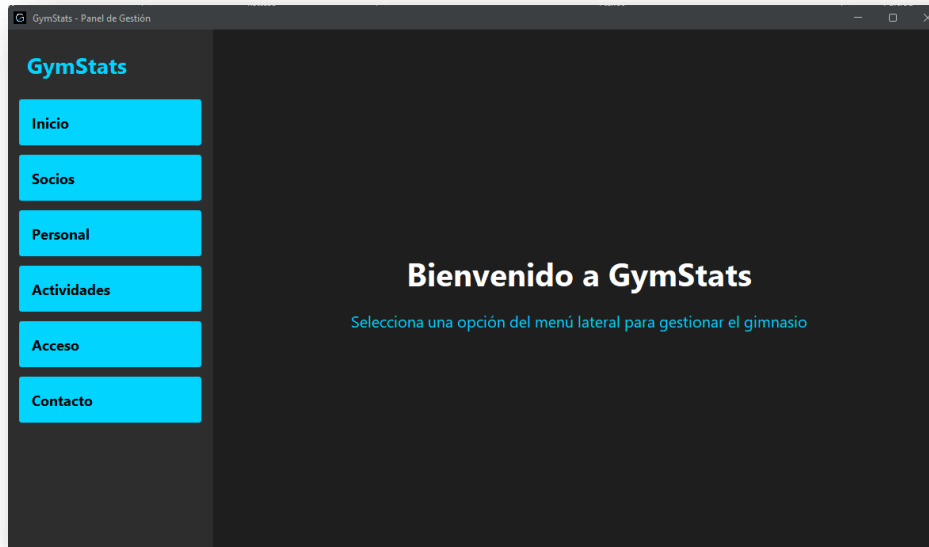


Paneles de Gestión Interna

El corazón del programa reside en las clases PanelAdministrador y PanelEmpleado. Ambos paneles comparten una estructura base compuesta por una barra lateral de navegación y un área de contenido dinámico. El administrador tiene acceso total a las pestañas de Socios, Personal, Actividades, Accesos y Mensajes de Contacto. Por su parte, el PanelEmpleado reutiliza gran parte de esta lógica pero oculta el acceso a la gestión de personal y a los contactos web por motivos de jerarquía. Visualmente, la



información se despliega mediante componentes **JTable** alimentados por un modelo de tabla que se actualiza realizando sentencias **SELECT** al servidor.





GymStats				
Gestión de Personal				
Añadir	Modificar	Eliminar	Actualizar	
DNI	Nombre	Apellido	Teléfono	Nómina
11111111A	Carlos	Sánchez	600111222	1500.0 €
22222222B	Laura	Gómez	600333444	1200.0 €
33333333C	Pedro	Ruiz	600555666	1100.0 €
44444444D	Ana	Martínez	600777888	1800.0 €
55555555E	Sergio	Fernández	600999000	1600.0 €

Módulos Funcionales de Gestión

Dentro de los paneles de administración y empleado, la aplicación despliega diferentes módulos especializados para cubrir las necesidades operativas del gimnasio. Cada uno de estos apartados interactúa de forma directa con un conjunto específico de tablas de la base de datos.

Gestión de Actividades

GymStats						
Gestión de Actividades						
Añadir Clase	Eliminar Clase		Actualizar			
DNI Ent.	Cód. Clase	Entrenador	Actividad	Zona	Hora	Descripción
22222222B	504	Laura	Zumba	Peso Libre	09:00	Sesión de Zumb...
11111111A	502	Carlos	Crossfit	Musculación	10:00	Clase intensa de ...
44444444D	501	Ana	Yoga	Funcional	18:00	Yoga para principi...
55555555E	503	Sergio	Pilates	Cardio	19:30	Pilates avanzado
33333333C	505	Pedro	Boxeo	Estiramientos	20:00	Entrenamiento d...



Este módulo es el encargado de organizar la agenda deportiva del centro. A través de esta interfaz, el personal puede visualizar y planificar las clases dirigidas que se imparten, como Zumba, CrossFit o Yoga. La tabla de esta ventana es el resultado de una consulta compleja que une la información de los Entrenadores, las Clases y las Zonas de Entreno, representando de forma visual la relación ternaria de nuestra base de datos. Desde aquí, los trabajadores pueden programar una nueva actividad asignando un instructor, un horario y un espacio específico, o eliminar sesiones canceladas.

Control de Accesos

Control de Accesos					
Actualizar		Modificar			
DNI	Nombre	Apellido	Fecha	Hora Entrada	Hora Salida
99999999Z	Juan	Pérez	28/05/2026	13:23	Dentro
55555555V	Marta	Ramos	27/10/2023	09:30	11:00
66666666W	Sara	Díaz	26/10/2023	11:00	12:30
77777777X	Luis	Torres	26/10/2023	08:45	10:30
88888888Y	Maria	López	25/10/2023	10:30	12:00

El apartado de accesos funciona como el registro de presencia del gimnasio. Muestra un listado ordenado cronológicamente con todos los fichajes de los socios. La lógica de este módulo cruza las tablas de Registrar_Entrada y Registrar_Salida. Como detalle de usabilidad, si un cliente ha fichado su entrada pero todavía no ha registrado su salida, el sistema procesa ese valor nulo y muestra automáticamente la palabra "Dentro" en la pantalla. Además, permite a los administradores y empleados modificar manualmente las horas de entrada y salida para corregir posibles olvidos de los usuarios.



Mensajes de Contacto (Integración Web)

The screenshot shows a web application window titled 'GymStats - Panel de Gestión'. On the left is a sidebar with navigation buttons: 'Inicio', 'Socios', 'Personal', 'Actividades', 'Acceso', and 'Contacto'. The main area is titled 'Mensajes de Contacto' and contains an 'Actualizar' button and a table of contact messages. A tooltip above the table says 'Haz doble clic en una fila para leer el mensaje completo'. The table has four columns: ID, Email, Teléfono, and Tipo de cliente. It contains four rows of data.

ID	Email	Teléfono	Tipo de cliente
4	lucas.fitness@email.com	633444555	Particular
3	info@suplementosgym.es	600123456	Empresa
2	marta.gomez@email.com	622333444	Particular
1	juan.interesado@email.com	611222333	Particular

Este módulo es exclusivo del Panel del Administrador por motivos de privacidad corporativa y actúa como puente de conexión entre el software de escritorio y la página web pública del gimnasio. Cuando un cliente potencial o una marca comercial rellena el formulario de información en la web, los datos se insertan en la tabla aislada Mensajes_Contacto. Desde la aplicación Java, el administrador recibe un listado de estas solicitudes indicando si provienen de un particular o una empresa. Al hacer doble clic sobre cualquiera de las filas, el sistema captura el ID del mensaje, realiza una consulta a la base de datos y despliega una ventana emergente de lectura con el texto completo y los datos de contacto.



Formularios de Operaciones CRUD

GymStats - Añadir Socio

NUEVO SOCIO

DNI del Socio

Nombre

Apellido

Usuario (Login)

Contraseña

GUARDAR SOCIO

Cancelar

GymStats - Programar Actividad

NUEVA ACTIVIDAD

Entrenador
11111111A - Carlos

Tipo de Clase
501 - Yoga

Zona de Entreno
1 - Cardio

Hora (HH:MM)

Descripción breve

PROGRAMAR CLASE

Cancelar

GymStats - Añadir Empleado

NUEVO EMPLEADO

DNI del Empleado

Nombre

Apellido

Teléfono

Usuario

Contraseña

Nómina (€)

Puesto
Entrenador

Especialidad / Turno

GUARDAR EMPLEADO

Cancelar



Para interactuar con la base de datos sin escribir código SQL de forma manual, hemos diseñado diversas ventanas emergentes que actúan como formularios. Las clases orientadas a la inserción, como AñadirSocio, AñadirEmpleado y AñadirActividad, recogen los datos tecleados, validan que los campos obligatorios no estén vacíos y ejecutan sentencias INSERT. De forma complementaria, las clases de modificación rescatan los datos previos de una fila seleccionada, los cargan en la pantalla para que el usuario los edite y finalizan la operación lanzando una sentencia UPDATE a PostgreSQL.

Panel del Cliente (Mi Perfil)

GymStats - Mi Perfil

¡Hola, Juan!

ESTADO DE TU MEMBRESÍA

Te quedan 0 días de acceso

TU IMC ACTUAL

22,5

NORMAL

CALCULADORA

Peso (kg)

Altura (cm)

ACTUALIZAR

GymStats

Finalmente, el PanelUsuario es la interfaz dedicada en exclusiva a los clientes del centro deportivo. Su lógica principal realiza consultas de solo lectura para mostrar la membresía actual y calcular internamente los días restantes de acceso. Como funcionalidad interactiva, incluye una herramienta de cálculo de Índice de Masa Corporal. Al introducir el peso y la altura, la clase aplica la fórmula matemática, indica



el estado de salud visualmente en pantalla y guarda de forma automática el nuevo valor del IMC en el perfil del cliente.

Pruebas de Software (Testing)

Durante el desarrollo, hemos realizado pruebas exhaustivas en ambos entornos. En la aplicación Java, se validó el CRUD intentando introducir letras en campos numéricos (como la nómina o el peso) o dejando campos obligatorios vacíos (DNI, Nombre). El sistema responde bloqueando la inserción y lanzando un JOptionPane con mensajes de error. A nivel web, implementamos expresiones regulares (Regex) en JavaScript para interceptar el formulario de contacto antes de que el código PHP actúe, comprobando que el email contenga un "@" válido y que el teléfono tenga el formato correcto, protegiendo así la base de datos contra datos basura.

Base de Datos

Arquitectura y Diseño de la Base de Datos

Para dar soporte a la aplicación de escritorio y al portal web, se ha implementado una base de datos relacional robusta en PostgreSQL. El diseño cubre los distintos ámbitos operativos del centro deportivo: control de trabajadores, gestión de clientes, distribución de zonas y planificación de clases.

Diseño ER

Nuestra base de datos está diseñada para la gestión de Gimnasios, con diferentes elementos que pueden tener los mismos, se contemplan varios ámbitos, como los trabajadores, clientes, zonas, clases, etc.

Primero creamos la entidad cliente, en la cual almacenaremos los atributos nombre, apellido y su DNI, los clientes compran membresías, también guardaremos la fecha donde el cliente compra la membresía. Una membresía puede ser comprada por muchos clientes y un cliente puede comprar solo una membresía.

De los clientes con membresía nos interesa tener un registro de entrada y uno de salida, cada uno queremos guardar la fecha y hora, cada registro de acceso guardaremos un código identificativo.

Muchos clientes con diferentes membresías pueden calcular en la calculadora. De la calculadora queremos almacenar el resultado del Id y el IMC.

Muchos clientes con muchas membresías pueden asistir a clases, de las cuales queremos guardar el código y tipo de clase.

Un empleado o varios pueden atender a varios clientes cada vez que se atiende a un cliente se le guarda la descripción de la consulta y la fecha de la misma, para mejorar los servicios del gimnasio.



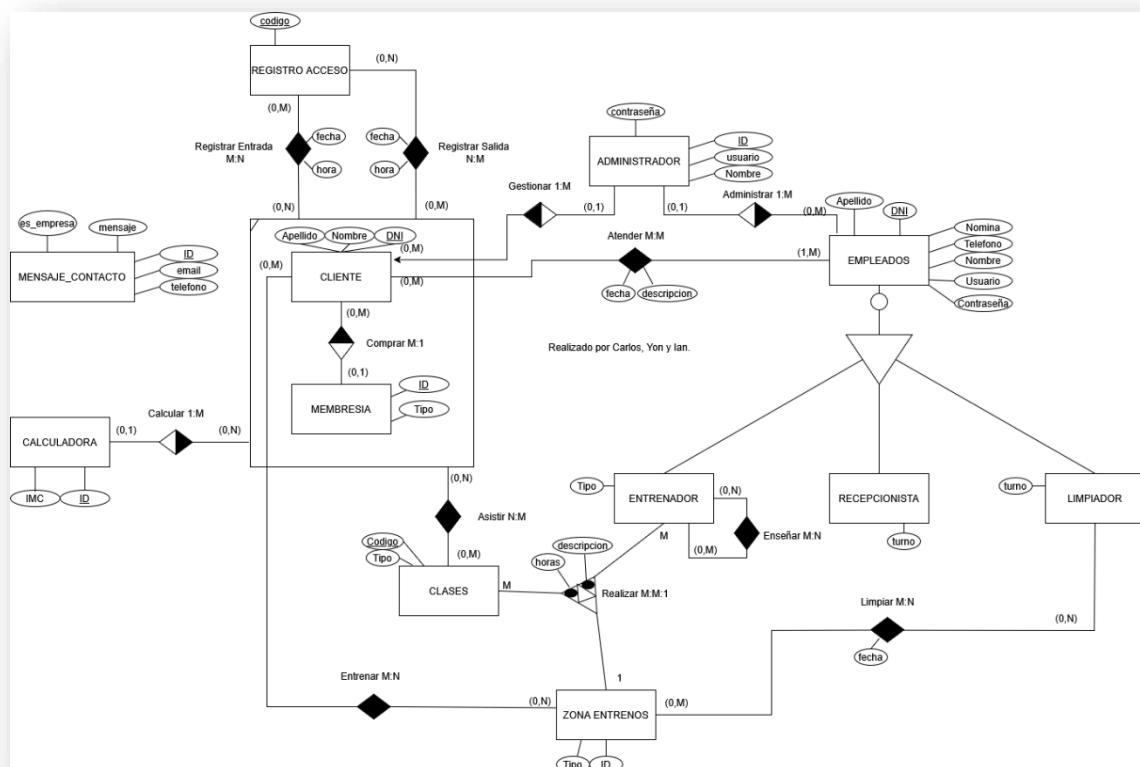
Hay varios tipos de empleados, de los empleados queremos guardar el DNI, el teléfono, el nombre, el apellido y la nómina. En total hay 4 tipos de empleados: los administradores, de los cuales guardaremos el usuario y contraseña de acceso; los recepcionistas, de los cuales guardaremos el turno; los limpiadores que también guardaremos el turno día/noche y los entrenadores, de los cuales guardaremos el tipo (artes marciales, entrenadores de fuerza, entrenadores de cardio...).

Un entrenador puede enseñar a muchos entrenadores y un entrenador puede ser enseñado por muchos entrenadores.

Una clase en una zona de entreno es realizada por muchos entrenadores, mientras que muchos entrenadores realizan una clase en una zona de entreno...

Muchos limpiadores limpian en muchas zonas de entreno, de esta acción almacenamos la fecha.

Por último, contamos con Mensajes de contacto independiente de las otras tablas. En ella almacenamos el identificador, email, teléfono, mensaje y si es una empresa. Su objetivo es actuar de forma aislada como un buzón de entrada desde la web pública para clientes potenciales y solicitudes externas.



Más Información

[Click aquí](#) para ver la memoria específica de la Base de Datos



Lenguaje de Marcas

El portal corporativo de GymStats está desarrollado con HTML5 semántico, estructurando la información en apartados clave como el inicio, las tarifas, la historia de la empresa y un catálogo de rutinas deportivas. Su diseño visual emplea una arquitectura CSS responsiva basada en Flexbox y Grid, garantizando una visualización perfecta tanto en dispositivos móviles como en monitores de escritorio, e incluye un tema oscuro nativo para modernizar la interfaz.



La interactividad del sitio se controla íntegramente mediante JavaScript, cumpliendo con los requisitos de validación y dinamismo exigidos en el proyecto. El script principal gestiona un sistema de traducción bilingüe y el cambio de tema visual guardando las preferencias del usuario en la memoria local del navegador. Además, aplica expresiones regulares en el formulario de contacto para validar estrictamente la estructura de los correos electrónicos y la longitud de los números de teléfono antes de permitir el envío de los datos.



Contáctanos

Correo Electrónico:

Número de Teléfono:

Mensaje:

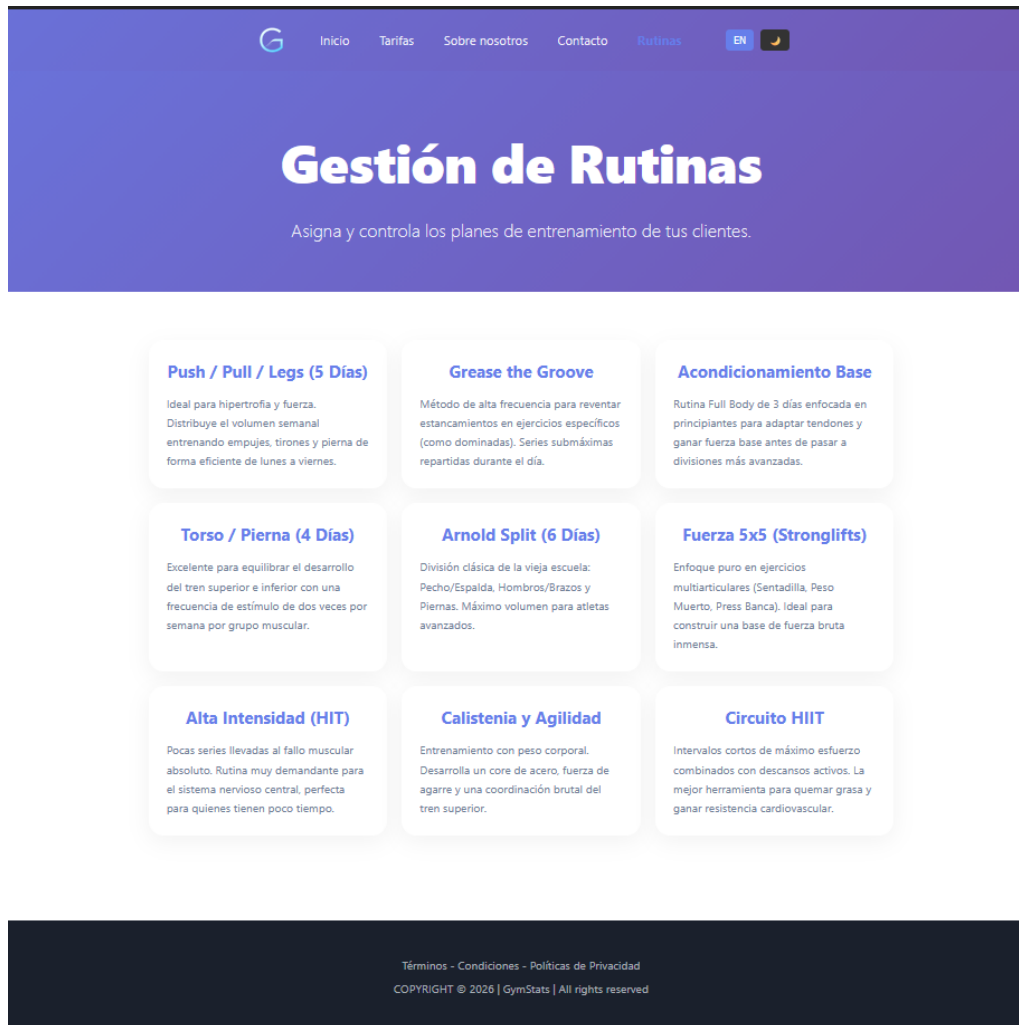
¿Eres una empresa? ☐

Enviar Solicitud

COPYRIGHT © 2026 | GynStats | All rights reserved

A nivel de integración de datos, la web se comunica con la base de datos PostgreSQL mediante una API construida en PHP. El sistema realiza una lectura en tiempo real lanzando una petición asíncrona que cuenta el número de usuarios registrados y expone la cifra en la página principal. Finalmente, el portal ejecuta una operación de escritura segura para recoger los mensajes del formulario e insertarlos directamente en la tabla de contacto, permitiendo que el administrador los lea desde la aplicación Java.

También tenemos un enlace en el header que linkea a la página de rutinas en la que hay ciertas rutinas que los usuarios pueden ver para inspirarse en sus entrenamientos:

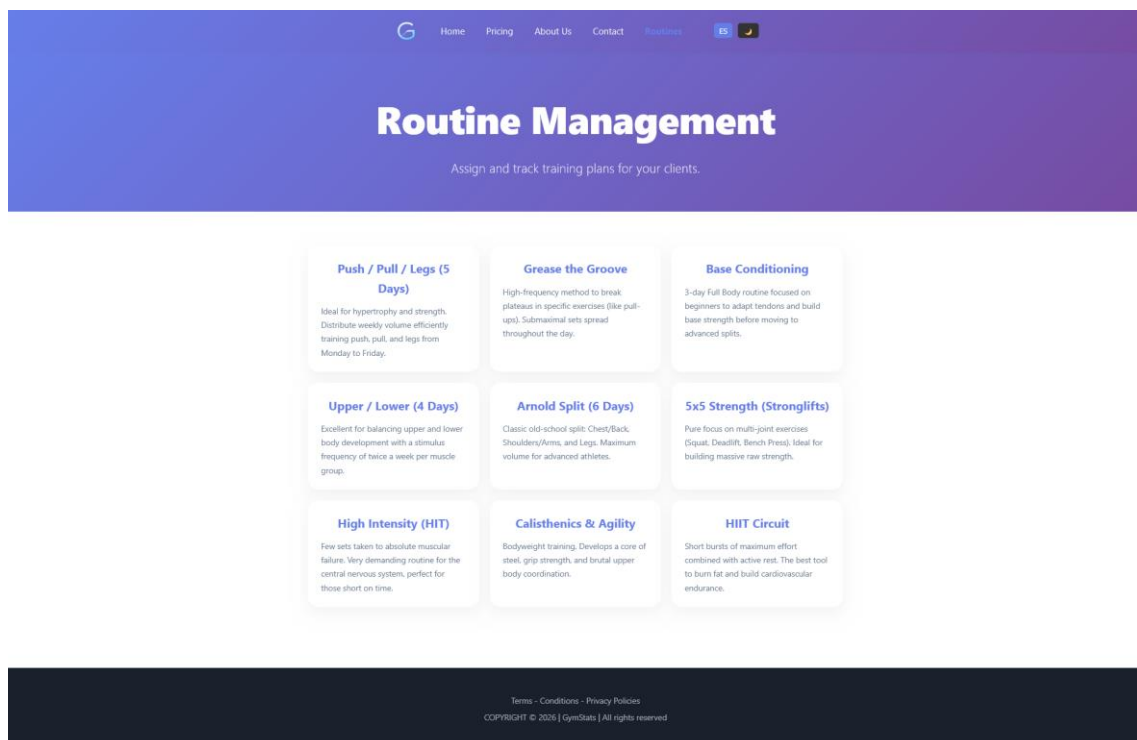
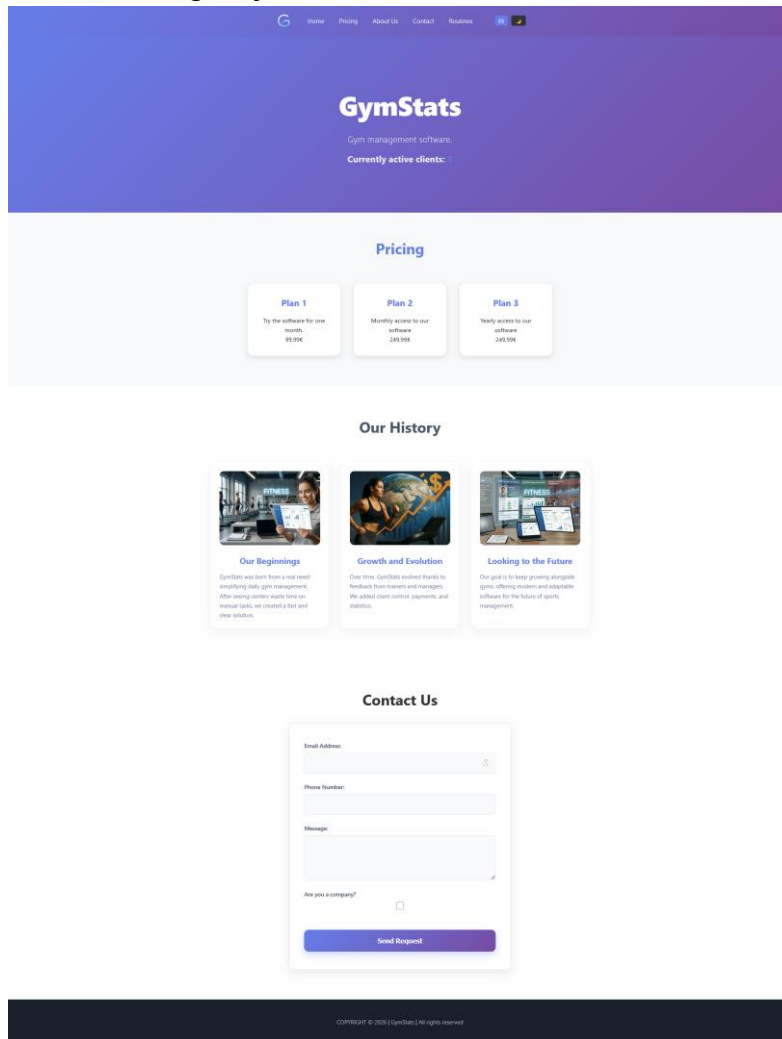


Botón inglés

Implementamos un botón para traducir el contenido de la página web al inglés. Esta funcionalidad es clave para que los usuarios extranjeros puedan navegar y comunicarse con nosotros sin barreras idiomáticas, abriéndonos las puertas como empresa a la



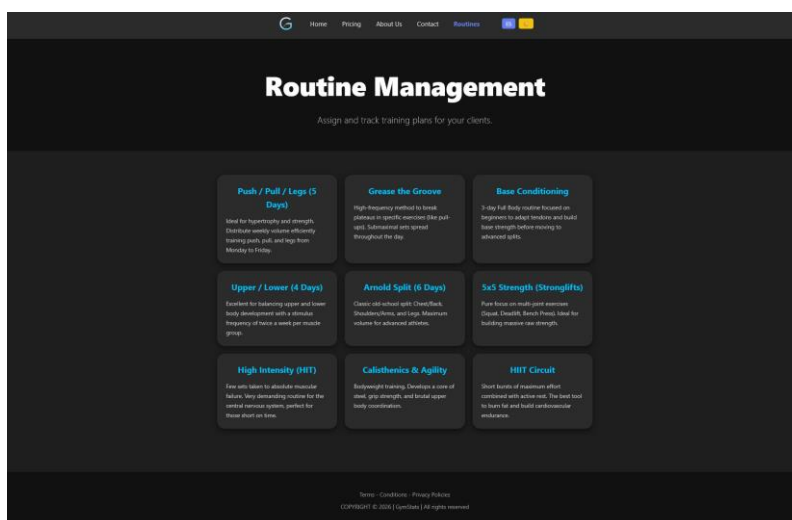
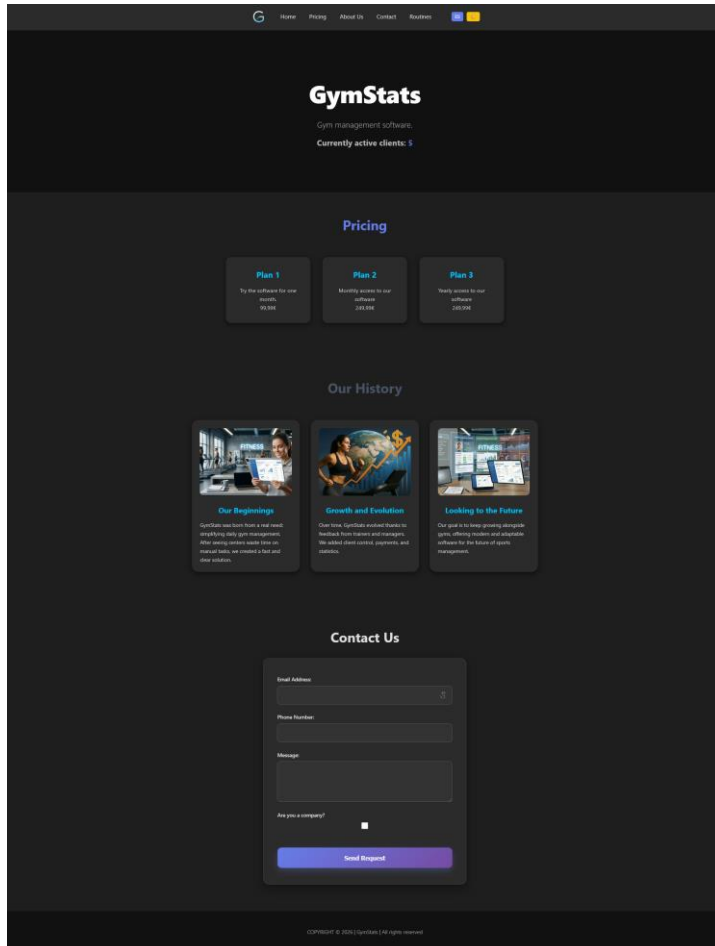
comunidad anglosajona e internacional.





Botón Dark Mode (Modo oscuro)

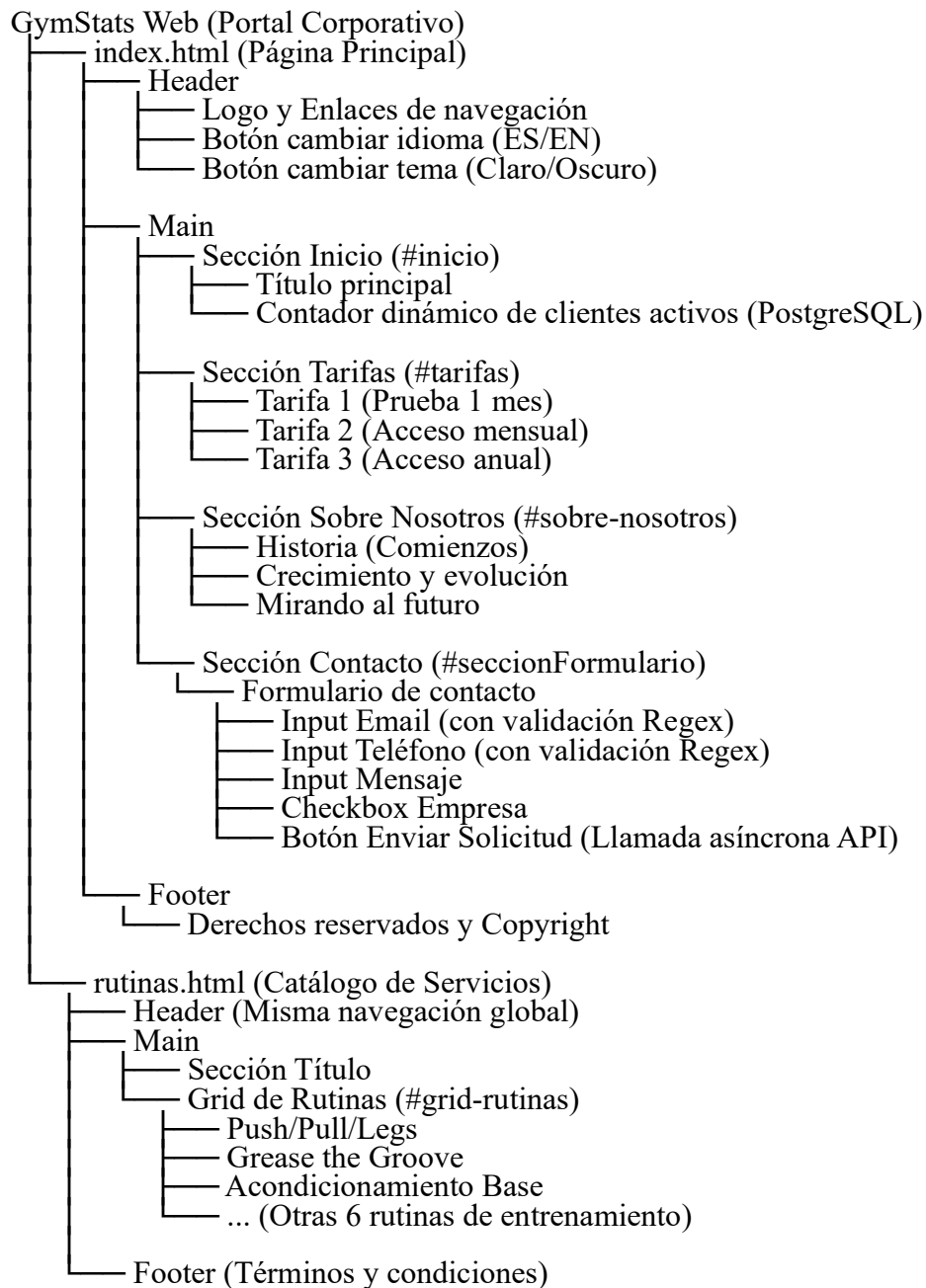
Usamos también un botón que permite alternar la interfaz entre un tema claro y uno oscuro (Dark Mode). Esta característica mejora notablemente la ergonomía y la experiencia de usuario, ya que reduce la fatiga visual cuando se navega en entornos con poca luz. Además, nos ayuda a mantener una coherencia estética con el diseño moderno y oscuro que hemos aplicado en nuestra aplicación de escritorio.





Prototipos

Mapa Navegación

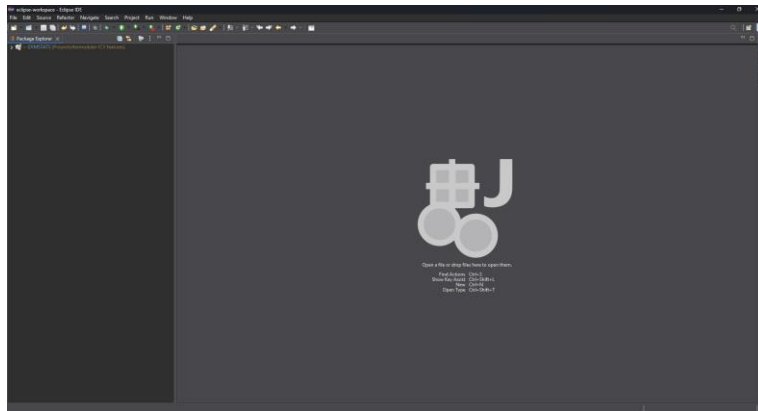


Entornos de desarrollo

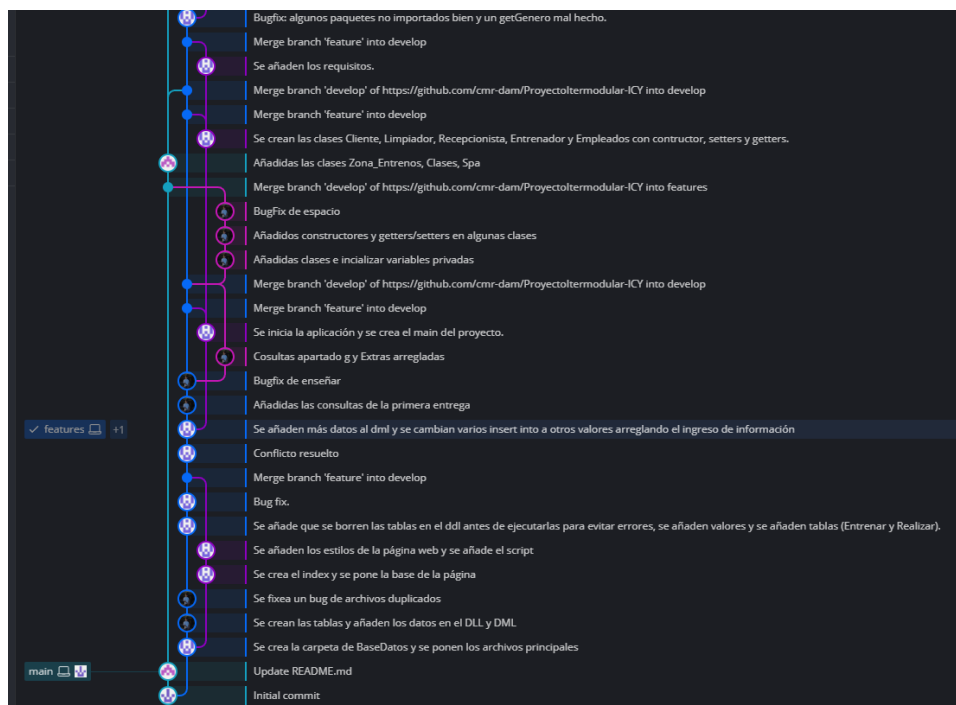
Para desarrollar GymStats nos hemos organizado de la forma más práctica y profesional posible. Toda la parte de la aplicación la hemos programado utilizando el IDE Eclipse, ya que nos facilitaba bastante el diseño de las ventanas y la estructura del proyecto. Todo el código que hemos hecho, desde el Java y la web hasta los scripts de la base de datos y esta misma documentación, está guardado a buen recaudo en un repositorio de



GitHub. Durante estas semanas hemos ido subiendo los cambios poco a poco, haciendo commits con mensajes descriptivos para saber qué iba añadiendo o arreglando cada uno sin volvernos locos.



Uso del git:

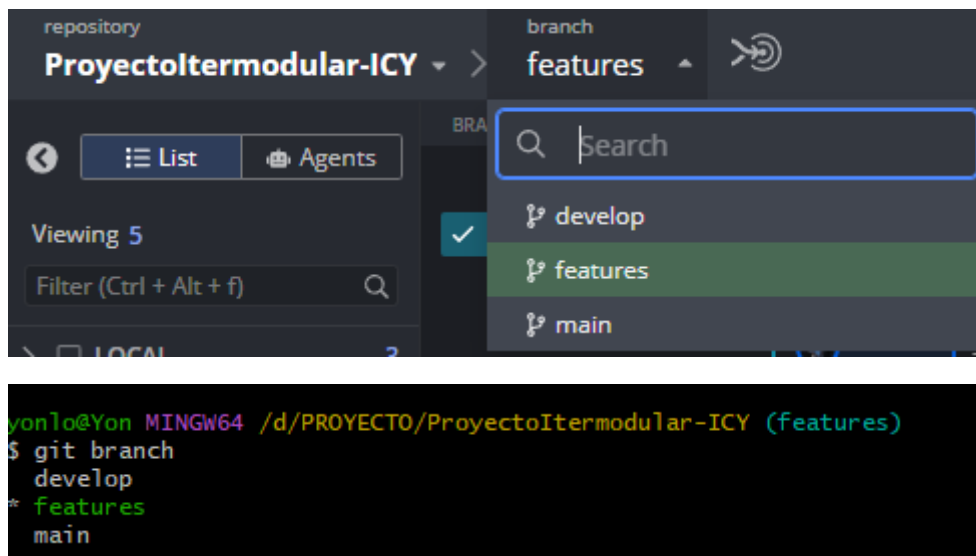




```
Yonlo@Yon MINGW64 /d/PROYECTO/ProyectoIntermodular-ICY (features)
$ git log --graph --pretty=format:'%C(yellow)%h%Creset %s %Cgreen(%cr)%Creset' --abbrev-commit --all
*33909a Se crea el applog.sql, archivo con los triggers, vistas y funciones. (7 hours ago)
*1444715 Resueltos conflictos al hacer merge (31 hours ago)
*497adf7 Se crea la ventanaEmpleado, se modifica el código de panelLogin para que se pueda iniciar sesión como empleado. Se añaden los atributos usuario y contraseña. (31 hours ago)
*6c5070 Se arreglan 2 fallos en el código, de conexión.php (31 hours ago)
*012bba Añadimos el logo a la app entera (2 days ago)
*277976 Se cambia la base de datos al ddl, consultas, y se cambia la memoria para que cuadre con las modificaciones de la base de datos (6 days ago)
*cf7e78 Actualizada la presentación (70% hecho) (6 days ago)
*9ef439f Merge branch 'develop' of https://github.com/cmr-dam/ProyectoIntermodular-ICY into develop (6 days ago)
*4d8119f Arreglado el método de registrarAcceso y añadirSalida y entrada. También ahora al cuando un usuario inicia sesión de nuevo se cambia automáticamente el rol. Se crea la ventana Actividades y acceso se pasa a controlar acceso de clientes, se crean los botones de crear y modificar actividad. Creada la clase ModActividad. (6 days ago)
*0950af4 Se cambian los las ips para que se conecten a la base de datos de las vm (6 days ago)
*475e2fe Resueltos conflictos de merge aceptando el borrado de bin (6 days ago)
*0c7226e Se añade el gitignore con los archivos de bin porque dan problemas con el git (7 days ago)
*cf15f817 Se añade la interfaz de control de accesos, se modifica el método cargar información para que muestre los accesos y se retocan algunos errores (6 days ago)
*7d847e9 Se añaden mensajes contacto a la app y se cambian los estilos de la misma (7 days ago)
*1b6a3b1 Se cambia la conexión con la bdd, se añade consulta de clientes en la página, se hace responsive, se añade el darkmode.css y se añade código en script.js (7 days ago)
*11e240e Se arregla el que el inc salga en la tabla de el panel administrador. También se repara el que el sistema cuando inicies sesión se guarde la fecha en la tabla de usuarios (7 days ago)
*5493bf1 Añadir presentación (medio terminada) a la nueva carpeta (6 days ago)
*81bafee El panel de inicio ahora se muestra al inicio, arreglados algunos bugs a la hora de cargar la bdd y quitados algunos imports sobrantes (8 days ago)
*2c0d4d8 Mejorado el estilo css, arreglados algunos bugs con id y clases y algunos fallos con modo oscuro (8 days ago)
*790681c Borradas clases inútiles, creada la lógica de los botones laterales y ventana de Gestionar empleados creada y funcional, a falta de el botón de modificar (8 days ago)
*cf8c4f0 Se cambia la página web, se quita clases y se añade rutinas.html, se quita el contador de clientes, se aclara el js y se arreglan varios bugs de la base de datos (8 days ago)
*3572aac Se completa lógica de botón de eliminar, se añaden las filas inc y ultimo_acceso a la bdd. Se añade un botón de actualizar que actualiza los parámetros de la bdd (8 days ago)
*4ecfbcd Añadido método modificar y creada ventana de modificar socio, añadidos los serialVersion a los usuarios para prevenir errores de version. Arreglado vacío de la bdd (8 days ago)
*07a0e67 Añadida la clase de AñadirSocio, la cual es una ventana en la cual se pueden crear socios y se actualiza en la bdd. También arreglados algunos bugs y añadidos algunos estilos (8 days ago)
*070bfef Añadido de IPE Finalizado (2 weeks ago)
*b3d802c Se añade nueva tabla al ddl, completa el script y se añade la api (2 weeks ago)
*d8f22f4 Se crea la tabla Administrador en la BDD, se crea el método para actualizar la tabla de el Panel Administrador según los clientes de la bdd, también se crea la clase Administrador (2 weeks ago)
*4968a4 Creada clase abstracta de usuario y se crean 2 extends a Cliente y Administrador. Se hace operativo el botón de login el cual ya hace conexión con base de datos (2 weeks ago)
*14c0497 Conexión con base de datos realizada mediante un getter de clase main, se instala el driver de Postgres en el proyecto y se optimiza el estilo visual (3 weeks ago)
*e993e91 Añadida la interfaz de usuario (pendiente base de datos) (4 weeks ago)
*75a21d2 Se añade la interfaz de administrador (4 weeks ago)
*fd187a3 Añadida la interfaz de usuario (pendiente base de datos) (4 weeks ago)
*ed0515d Añadida la interfaz de usuario (pendiente base de datos) (4 weeks ago)
*3d8edcb añadidas class a gitignore (4 weeks ago)
*88b6a3 Quitadas clases inútiles, creado interfaces usuario y administrador (4 weeks ago)
*63b0da0 Interfaz mejorada y añadidas validaciones en el botón de inicio de sesión (6 weeks ago)
*4f8316a Añadido título a la ventana principal, título, colores y panel (6 weeks ago)
*534f836 Retirado el paquete main y dejamos el por defecto. También se arreglan algunos archivos duplicados que provocaban errores (6 weeks ago)
*b61477c Bugfix en merge arreglado (3 months ago)
*9fd851c Merge branch 'develop' of https://github.com/cmr-dam/ProyectoIntermodular-ICY into develop (3 months ago)
*cf9bc4b Se hacen cambios en las clases Calculadora, Membresía, Registro acceso, spa, vestuario y ZonaEntrenos (3 months ago)
*657a2dd Se cambia la clase Gestion, se añade tema y se crea panel de botones (3 months ago)
*2386014 Bigfix module-info (3 months ago)
*5f77d67 Inicializada interfaz grafica (3 months ago)
*4d6b291 Merge branch 'develop' of https://github.com/cmr-dam/ProyectoIntermodular-ICY into develop (3 months ago)
*a31d478 Se recorta y se quita el fondo del logo, se añade a la página web. (3 months ago)
*67b4a59 Merge branch 'develop' of https://github.com/cmr-dam/ProyectoIntermodular-ICY into develop (3 months ago)
*738fe3b Añadidos a href y generación de imágenes (3 months ago)
*15bc72c Se añaden los logos editados de GYMSTATS (3 months ago)
```

Al ser tres (Carlos, Yon e Ian), programar a la vez el mismo proyecto tenía el peligro de que nos pisáramos el código unos a otros. Para evitar machacarnos el trabajo, hemos seguido la filosofía GitFlow. La norma básica era que la rama Main era sagrada: solo se ha utilizado para dejar las versiones estables y la entrega final del programa.

El trabajo del día a día lo hacíamos en las ramas Feature. Cuando a uno le tocaba implementar algo nuevo, se creaba su propia rama aislada. Ahí programábamos y hacíamos nuestras pruebas. Cuando veíamos que la funcionalidad estaba terminada y no rompía el resto del programa, entonces la integrábamos en la rama Develop.

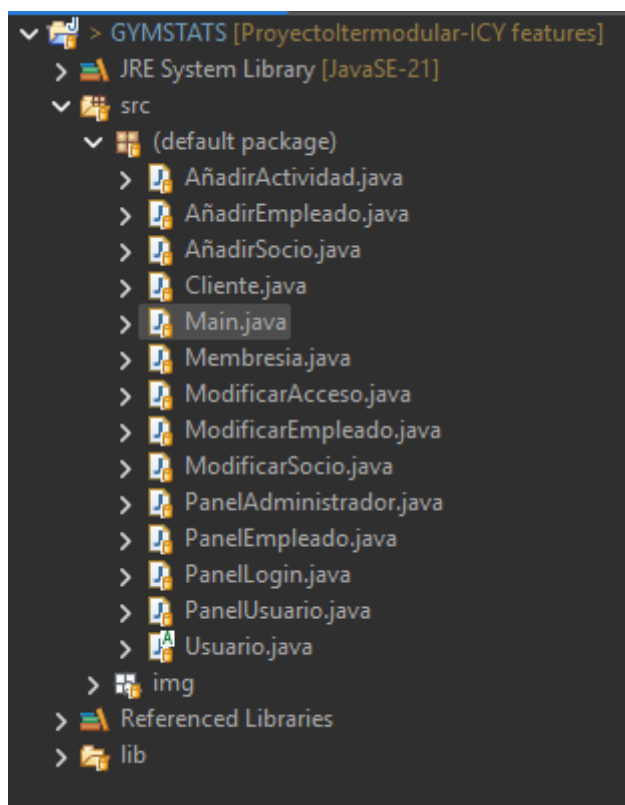




Básicamente, la rama Develop ha sido nuestro punto de encuentro para sincronizar el código de los tres. Es cierto que a veces, por las prisas o por corregir algún fallo rápido, hemos podido hacer algún commit directamente en Develop por error, pero por lo general hemos respetado el flujo de trabajo aislando los desarrollos en las ramas Feature. Gracias a organizarnos así, hemos evitado conflictos raros en Git y el proyecto ha salido adelante de forma muy ordenada.

Implementación y Estructura del Proyecto

La implementación del proyecto se ha dividido en dos grandes bloques organizados de forma modular. Por un lado, la aplicación Java sigue un patrón arquitectónico simplificado. En el paquete principal encontramos las clases de modelo, como Cliente, Usuario y Membresia, que representan la lógica de negocio y almacenan los datos temporalmente. Las vistas y controladores están unificados en las clases de interfaz, destacando Main para el arranque, PanelLogin para la autenticación, y los distintos paneles de gestión principal como PanelAdministrador y PanelEmpleado. Además, se incluyen ventanas emergentes como AñadirSocio o ModificarEmpleado que interactúan directamente con la conexión JDBC a PostgreSQL.



Por otro lado, la página web presenta una estructura de directorios clásica y limpia. En la raíz del servidor se ubican los archivos HTML principales como el index y la página de rutinas. De esta raíz cuelgan las carpetas de recursos: una carpeta para las hojas de estilo CSS, otra para la lógica de validación e internacionalización en JavaScript, el directorio de imágenes para los recursos gráficos, y finalmente la carpeta API, donde



reside el script PHP encargado de realizar la conexión segura con la base de datos para insertar los mensajes de contacto.



Dificultades y Soluciones Técnicas / Technical Challenges and Solutions

ES

A lo largo del desarrollo de GymStats nos hemos enfrentado a varios retos técnicos. Una de las mayores dificultades iniciales fue sincronizar el trabajo de los tres miembros del equipo mediante Git sin sobrescribir el código, problema que solucionamos adoptando de forma estricta la metodología GitFlow y aislando los desarrollos en ramas independientes. En la parte de la aplicación Java, el manejo de la biblioteca Swing resultó complejo a la hora de adaptar el diseño visual, lo cual resolvimos implementando la librería externa FlatDarkLaf para conseguir un acabado mucho más moderno y oscuro, ya que el apartado que nos aporta java de default no nos convencía.



Respecto a la base de datos, lidiar con las claves foráneas al intentar borrar registros nos generó errores. Esto nos obligó a rediseñar los scripts SQL de creación de tablas añadiendo la restricción de borrado en cascada para evitar bloqueos. Finalmente, en la página web, la conexión entre el formulario HTML y la base de datos requirió aprender a configurar sentencias preparadas en PHP y manejar peticiones asíncronas con la función Fetch de JavaScript para evitar recargas en la página y bloquear posibles inyecciones de código.

EN

Throughout the development of GymStats, we have faced several technical challenges. One of the major initial difficulties was synchronizing the work of the three team members via Git without overwriting code, a problem we solved by strictly adopting the GitFlow methodology and isolating developments in independent branches. In the Java application, handling the Swing library proved complex when adapting the visual design, which we resolved by implementing the external FlatDarkLaf library to achieve a much more modern and dark finish, as we were not convinced by Java's default appearance.

Regarding the database, dealing with foreign keys when attempting to delete records caused errors. This forced us to redesign the SQL table creation scripts by adding the cascade delete constraint to prevent blocks. Finally, on the website, the connection between the HTML form and the database required us to learn how to configure prepared statements in PHP and manage asynchronous requests using the JavaScript Fetch function to avoid page reloads and block potential code injections.

```
DROP TABLE IF EXISTS Mensajes_Contacto CASCADE;
DROP TABLE IF EXISTS Entrenar CASCADE;
DROP TABLE IF EXISTS Limpiar CASCADE;
DROP TABLE IF EXISTS Enseñar CASCADE;
DROP TABLE IF EXISTS Realizar CASCADE;
DROP TABLE IF EXISTS Atender CASCADE;
DROP TABLE IF EXISTS Asistir CASCADE;
DROP TABLE IF EXISTS Registrar_Salida CASCADE;
DROP TABLE IF EXISTS Registrar_Entrada CASCADE;
DROP TABLE IF EXISTS Cliente CASCADE;
DROP TABLE IF EXISTS Limpiador CASCADE;
DROP TABLE IF EXISTS Recepcionista CASCADE;
DROP TABLE IF EXISTS Entrenador CASCADE;
DROP TABLE IF EXISTS Administrador CASCADE;
DROP TABLE IF EXISTS Empleados CASCADE;
DROP TABLE IF EXISTS Zona_Entrenos CASCADE;
DROP TABLE IF EXISTS Clases CASCADE;
DROP TABLE IF EXISTS Registro_Acceso CASCADE;
DROP TABLE IF EXISTS Calculadora CASCADE;
DROP TABLE IF EXISTS Membresia CASCADE;
```



Selección de Colores y Temas

Elegimos los colores negro y azul porque quedan minimalista y futuristas, creemos que combinan muy bien con la temática de software que es tecnológico y a demás nos hace diferenciarnos del resto de empresas en relación con el mismo ámbito.

Usamos “FlatLaf - Flat Look and Feel” porque nos da un diseño integrado como librería muy minimalista, actualizando los antiguos visuales de java swing.

Para la web usamos colores blancos, azules y morados en modo claro y en modo oscuro lo mismo que la app para estar en sintonía, de forma que el usuario siente que somos un grupo organizado que tenemos en cuenta el trabajo de nuestros compañeros.

Reparto de Tareas y Trabajo en Equipo

El desarrollo del proyecto ha sido un esfuerzo conjunto donde todos los miembros han participado de forma activa, organizando y gestionando el flujo de trabajo de forma ágil mediante un tablero en Canva. Aunque todos hemos colaborado en la depuración de errores y el manejo del control de versiones con Git, el peso del proyecto se ha repartido según las áreas de cada uno para ser más eficientes.

Carlos ha liderado el desarrollo de la aplicación de escritorio en Java, asumiendo la mayor parte de la programación del código (aunque con apoyo del resto del equipo), y se ha encargado íntegramente de la configuración y despliegue de las máquinas virtuales. Yon ha asumido el diseño y desarrollo del portal web corporativo, además de encargarse principalmente de la redacción, maquetación y estructuración de esta memoria técnica, contando con la revisión y ayuda puntual de sus compañeros. Por su parte, Ian ha trabajado en la documentación correspondiente al módulo de IPE enfocada en la gestión de contratos, ha preparado el material para la presentación oral del proyecto y ha desarrollado la programación avanzada de la base de datos, encargándose



de los triggers, las vistas y las funciones.

MEJORAS 26/05	FORMATO
Parte General ✓✓ Crear la exposición en canva ✓✓ Crear memoria de base de datos ✗✗ Crear memoria del proyecto intermodular ✓ Mejorar el README.md Parte IPE ✓✓ Creación de roles ✓✓ Documento informativo ✓ Sitio de trabajo ✓ Jornada / horario / sueldo ✓ Beneficios de conciliación ✓ Identificación de riesgos ✓ Contratos de trabajo ✓ Tipo de contrato ✓ Creación del modelo de contrato Parte APP ✓✓ Implementación de un main básico ✓ Creación de todas las clases básicas con su constructor, setters y getters ✓ Reducción de las clases ✓ Añadir panel admin ✓ Añadir panel usuarios ✓ Añadir login de admin y usuarios ✓ Acabar el apartado de Control de acceso en admin ✓ Reparar que si un socio calcula el imc aparezca en la tabla de admin y no salga error (problema principalmente creo que de la base de datos) ✓ Poner un icono a la app ✓ Poner el texto de Usuario/Contraseña incorrecto más grande ✓ Acabar el apartado de Gestionar actividades en admin ✓ Hacer panel de Empleados ✓ Cambiar el login para que los empleados accedan también	✓ Tarea realizada ✗ Tarea pendiente ✗(Nombre) Tarea pendiente asignada ✗✗/✓✓ Tarea pendiente/realizada con mayor importancia
	ENLACES
	GitHub Memoria General Memoria base de datos Memoria IPO

Despliegue y Sistemas Informáticos

Para el entorno de producción, hemos diseñado una arquitectura distribuida basada en la creación y configuración de dos máquinas virtuales independientes utilizando Ubuntu Server. La primera máquina virtual actúa exclusivamente como servidor de base de datos, alojando el motor PostgreSQL 16. La segunda máquina virtual está dedicada al servidor web corporativo, corriendo Apache y PHP.

Para garantizar la comunicación estable en la red local, ambas máquinas cuentan con direcciones IP fijas (estáticas) configuradas desde el sistema Ubuntu. Además, se han habilitado y configurado las reglas del cortafuegos (UFW) en ambas y se ha modificado el archivo pg_hba.conf en el servidor de base de datos para permitir peticiones entrantes remotas.

Finalmente, el cliente final (el host) es el equipo físico con Windows 11 de uno de los administradores. Desde este equipo se ejecuta la aplicación de escritorio Java (que conecta en remoto a la primera máquina) y se utiliza el navegador web para visualizar la página alojada en la segunda máquina. Todo el despliegue de tablas y datos se realiza de forma automatizada lanzando nuestros scripts DDL y DML directamente sobre el servidor PostgreSQL.



```
Ubuntu-SV-BBDD (Base de datos actualizada - web funcional) [Corriendo] - Oracle Virtu...
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
cmanzano@cmanzanosv:~$ sudo systemctl status postgresql
• postgresql.service - PostgreSQL RDBMS
  Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
  Active: active (exited) since Sat 2026-05-30 18:06:15 UTC; 47s ago
  Process: 1109 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
  Main PID: 1109 (code=exited, status=0/SUCCESS)
  CPU: 9ms

may 30 18:06:15 cmanzanosv systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
may 30 18:06:15 cmanzanosv systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
cmanzano@cmanzanosv:~$ _
```

```
Ubuntu-SV (Web actualizada y conexion bbdd) [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
cmanzano@cmanzanosv:~$ sudo systemctl status apache2
[sudo] password for cmanzano:
• apache2.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
  Active: active (running) since Sat 2026-05-30 18:06:11 UTC; 1min 1s ago
  Docs: https://httpd.apache.org/docs/2.4/
  Process: 1111 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 1138 (apache2)
  Tasks: 6 (limit: 4599)
  Memory: 30.0M (peak: 30.5M)
  CPU: 231ms
  CGroup: /system.slice/apache2.service
          └─1138 /usr/sbin/apache2 -k start
            └─1166 /usr/sbin/apache2 -k start
              └─1167 /usr/sbin/apache2 -k start
                └─1168 /usr/sbin/apache2 -k start
                  └─1169 /usr/sbin/apache2 -k start
                    └─1170 /usr/sbin/apache2 -k start

may 30 18:06:11 cmanzanosv systemd[1]: Starting apache2.service - The Apache HTTP Server...
may 30 18:06:11 cmanzanosv apachectl[1134]: AH00558: apache2: Could not reliably determine the se
may 30 18:06:11 cmanzanosv systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-20/20 (END)
```

Itinerario Personal para la Empleabilidad

[Click aquí](#) para ver el documento de IPE.

Conclusiones y Trabajo Futuro

La realización de este proyecto intermodular ha supuesto la consolidación técnica de todos los conocimientos adquiridos. Haber levantado desde cero un sistema que conecta



una interfaz de escritorio y una web pública con una misma base de datos centralizada demuestra la viabilidad de GymStats como producto real. Como trabajo futuro para seguir escalando la aplicación, tenemos previsto desarrollar una pasarela de pago para el cobro automático de las cuotas de las membresías directamente desde la web. También nos planteamos la creación de una aplicación móvil para que los clientes puedan llevar su tarjeta de acceso digital mediante un código QR, y la implementación de un panel de métricas con gráficos en el área del administrador para analizar visualmente la rentabilidad del gimnasio y las horas punta de asistencia.

Enlace al proyecto

[Click aquí](#) para entrar al proyecto de Github.